CLI-Bench: A Memorization-Proof Benchmark for Tool-Agnostic Command-Line Interface Orchestration

Minerva & Klik Team
tech@minervacap.cn

March 14, 2026

Abstract

We introduce CLI-BENCH, a benchmark for evaluating AI agents' ability to orchestrate command-line interface (CLI) tools to complete multi-step workflows. Unlike existing tool-use benchmarks that evaluate agents on fixed API sets likely encountered during pretraining, CLI-BENCH is designed to measure *genuine tool learning*—the capacity to read novel documentation and correctly invoke unfamiliar interfaces. Our key contribution is the inclusion of five **fictional CLI tools** (KForge, FlowCtl, MeshCtl, DataPipe, AlertMgr) with realistic but entirely novel command structures that no language model has seen in training data. The benchmark comprises 40 tasks across six categories (DevOps, project management, communication, data operations, custom CLI, composite workflows), evaluated against stateful mock backends with deterministic execution. We evaluate six frontier models (Claude Opus 4.5, GPT-5.2, Gemini 3.1 Pro, DeepSeek-V3.2, Llama 4 Maverick, Qwen 3.5) across 7,200 experimental runs. Our results reveal: (1) performance drops 14.0% on fictional tools compared to real tools—*larger* than the 13% gap observed in 2024 models, suggesting increased reliance on memorization as capabilities improve; (2) multi-tool composite tasks remain challenging at 58.7% for the best model; (3) pass^k consistency scores degrade from 78.6% (pass^1) to 52.8% (pass^5), a 33% relative drop; (4) open-source DeepSeek-V3.2 achieves 93% of frontier performance at 10% of the cost. CLI-BENCH is publicly available with full evaluation harness at `https://github.com/minervacap2022/CLI-Bench`.

1 Introduction

The deployment of large language models (LLMs) as autonomous agents capable of executing complex workflows has progressed rapidly [Liu et al., 2023, Zhou et al., 2023]. Modern LLM-based agents can browse the web, write code, and orchestrate API calls to accomplish user goals. However, evaluating these capabilities presents a fundamental challenge: how do we distinguish genuine *tool learning* from mere *tool memorization*?

Consider an agent that successfully executes `gh issue create -title "Bug fix" -body "Description"`. Has the agent learned to read GitHub CLI documentation and apply it correctly, or has it simply memorized this common command pattern from its training corpus? This distinction matters critically for deployment: an agent that has memorized GitHub CLI syntax may fail catastrophically when asked to use an unfamiliar internal CLI tool.

Existing tool-use benchmarks suffer from this conflation. ToolBench [Qin et al., 2023] evaluates on 16,000+ real-world APIs from RapidAPI—but these APIs and their documentation are publicly available and likely present in LLM training data. API-Bank [Li et al., 2023] uses a fixed set of APIs that models may have memorized. Even WebArena [Zhou et al., 2023], while providing realistic environments, tests on standard web interfaces that models have encountered extensively.

We introduce CLI-BENCH to address this gap through a novel design principle: **fictional tool evaluation**. Alongside seven real CLI tools (GitHub CLI, Slack CLI, Linear, Jira, Notion, Google Workspace, generic CRUD), we include

Table 1: Comparison with existing tool-use benchmarks. CLI-BENCH uniquely provides memorization-proof evaluation through fictional tools while maintaining tool-agnostic architecture.

Benchmark	Tool Agnostic	Fictional Tools	Stateful	pass^k	Tasks
ToolBench	✗	✗	$\sim$	✗	126K
API-Bank	✗	✗	✗	✗	314
AgentBench	✗	✗	✓	✗	1,014
τ-bench	$\sim$	✗	✓	✓	215
CLI-BENCH (Ours)	✓	✓	✓	✓	40

five fictional CLI tools—KForge (knowledge management), FlowCtl (workflow automation), MeshCtl (service mesh control), DataPipe (data pipeline orchestration), and AlertMgr (alerting and monitoring)—that follow realistic CLI conventions but implement entirely novel domain semantics. No language model has encountered these tools in training, providing a clean test of tool learning capability.

Contributions. We make the following contributions:

1. **Memorization-proof evaluation**: Five fictional CLI tools with novel command structures that no model has seen during training, enabling clean measurement of tool learning.
2. **Tool-agnostic architecture**: YAML-based tool adapter specifications enabling evaluation on any CLI without code changes to the harness.
3. **Stateful mock backends**: Deterministic service simulators with deep state-diffing for reproducible evaluation without live API dependencies.
4. **Comprehensive experimental analysis**: 1,200 runs across 5 models, with ablations on tool familiarity, task complexity, and error recovery.
5. **Reliability metrics**: Adoption of pass^k from τ-bench [Yao et al., 2024] to measure consistency across repeated runs.

2 Related Work

Tool-Use Benchmarks. ToolBench [Qin et al., 2023] pioneered large-scale API evaluation with 16,464 real-world APIs and introduced the DFSDT algorithm for solution path discovery. However, it relies on actual API calls, introducing non-determinism, rate limiting, and potential training data contamination. API-Bank [Li et al., 2023] provides structured evaluation but with a fixed, memorizable tool set. ToolAlpaca [Tang et al., 2023] and Gorilla [Patil et al., 2023] focus on API retrieval rather than multi-turn execution. Our work differs fundamentally by introducing fictional tools that cannot have been memorized.

Agent Benchmarks. AgentBench [Liu et al., 2023] evaluates agents across 8 environments but does not address tool learning versus memorization. SWE-bench [Jimenez et al., 2024] tests code editing on real GitHub issues, achieving ecological validity but domain-specific scope. GAIA [Mialon et al., 2023] evaluates general assistant capabilities with tasks designed to be simple for humans but challenging for AI, achieving a 77-point human-AI gap. τ-bench [Yao et al., 2024] introduced the pass^k metric we adopt, measuring reliability degradation across repeated trials in retail and airline domains.

Data Contamination. Recent work has highlighted the severity of data contamination in LLM evaluation. Jimenez et al. [2024] found that 60.83% of SWE-bench tasks had potential solution leakage. Studies on MMLU show GPT-4o dropping from 88% to 71.9% on contamination-free variants. Our fictional tool approach provides a principled solution: if a tool has never existed publicly, it cannot be in training data.

3 Benchmark Design

3.1 Design Principles

CLI-BENCH is built on three core principles:

1. **Memorization Resistance**: Fictional tools with novel semantics ensure evaluation of genuine tool learning, not pattern recall.
2. **Tool Agnosticism**: YAML adapter specifications enable evaluation on any CLI without harness modifications.
3. **Reproducibility**: Stateful mock backends provide deterministic execution, eliminating external API dependencies and rate limiting.

3.2 Tool Adapter Specification

Each tool is defined by a YAML adapter specifying its interface:

```
name: "kforge"
binary: "kforge"
description: "Knowledge management and document synthesis"
auth:
  type: "api_key"
  env_var: "KFORGE_API_KEY"
commands:
  - name: "doc"
    subcommands:
      - name: "create"
        description: "Create a new knowledge document"
        args:
          - name: "title"
            type: "string"
            required: true
        flags:
          - name: "--tags"
            type: "string"
            description: "Comma-separated tags"
          - name: "--template"
            type: "string"
            enum: ["blank", "meeting", "decision", "research"]
```

This specification serves dual purposes: (1) generating tool documentation provided to agents, and (2) defining command parsing for the mock backend.

3.3 Fictional Tool Suite

We designed five fictional CLI tools following realistic conventions:

Each fictional tool was designed by studying real analogues (Notion, Temporal, Istio, Airflow, PagerDuty) but implementing distinct command structures and semantics that do not appear in public documentation.

3.4 Task Structure

Each of the 40 tasks comprises:

- A natural language **instruction** describing the user's goal

Table 2: Fictional CLI tools in CLI-BENCH. Each tool follows standard CLI patterns but implements novel domain semantics.

Tool	Domain	Commands	Key Operations
KForge	Knowledge Management	8	doc create/link, search, synthesis
FlowCtl	Workflow Automation	12	flow create/trigger, step add, schedule
MeshCtl	Service Mesh	10	service register, route add, canary
DataPipe	Data Pipelines	11	pipe create, transform add, run
AlertMgr	Monitoring/Alerting	9	alert create, rule add, silence

- A list of **available tools** (as adapter references)
- An **initial state** for each relevant mock backend
- An **expected state** defining success criteria
- **Difficulty classification**: Easy (single-tool, 1-2 commands), Medium (multi-tool or error recovery), Hard (complex dependencies or orchestration)

3.5 Mock Backend Architecture

We implement stateful mock backends for each tool category:

- **GitHub**: Repositories, issues, pull requests, commits, labels
- **Slack**: Channels, messages, users, threads, reactions
- **Linear/Jira**: Issues, projects, workflows, sprints, assignments
- **Notion**: Pages, databases, blocks, properties
- **Google Workspace**: Calendar events, Drive files, Gmail messages
- **Fictional**: Generic CRUD backend supporting all five fictional tools

Backends parse CLI commands, update internal state, and return realistic stdout/stderr including error messages for invalid operations.

4 Evaluation Methodology

4.1 Execution Protocol

We adopt a multi-turn execution protocol where the agent receives:

1. The task instruction and full tool documentation (generated from YAML specs)
2. After each command: stdout, stderr, exit code, and remaining turn count

The agent produces either a `command` (executed against mock backends) or `finish` (terminates the episode). Maximum turns are task-specific (10 for easy, 15 for medium, 20 for hard).

4.2 Scoring Metrics

Outcome Score. We compute state similarity between actual and expected backend states:

$$\text{Outcome} = \text{DeepDiff}(S_{\text{actual}}, S_{\text{expected}}) \in [0, 1] \tag{1}$$

The DeepDiff algorithm recursively compares nested structures, awarding partial credit for partially correct states (e.g., correct issue title but wrong assignee scores 0.5 on that entity).

Efficiency Score. Rewards agents that solve tasks in fewer commands:

$$\text{Efficiency} = \min\left(1.0, \frac{C_{\text{optimal}}}{C_{\text{actual}}}\right) \tag{2}$$

where C_{optimal} is the minimum command count determined by human annotation.

Recovery Score. Measures error handling capability:

$$\text{Recovery} = \begin{cases} 1.0 & \text{if errors encountered and successfully recovered} \\ 0.5 & \text{if no errors occurred (baseline)} \\ 0.0 & \text{if errors occurred without recovery} \end{cases} \tag{3}$$

Composite Score. Weighted combination:

$$\text{Score} = 0.6 \cdot \text{Outcome} + 0.2 \cdot \text{Efficiency} + 0.2 \cdot \text{Recovery} \tag{4}$$

4.3 Pass^k Consistency Metric

Following τ-bench [Yao et al., 2024], we measure reliability using pass^k:

$$\text{pass}^k = \mathbf{1}\left[\forall i \in \{1, \ldots, k\} : \text{Outcome}_i \geq 0.5\right] \tag{5}$$

A task passes only if *all* k independent runs achieve outcome ≥ 0.5. This penalizes inconsistent agents that occasionally succeed but frequently fail. We report pass^1, pass^3, and pass^5 in our experiments.

5 Experiments

5.1 Experimental Setup

Models. We evaluate six frontier models representing the state-of-the-art as of March 2026:

- **GPT-5.2** (OpenAI, gpt-5.2-2026-02) – Latest GPT series with enhanced reasoning
- **Claude Opus 4.5** (Anthropic, claude-opus-4-5-20251101) – 80.9% on SWE-bench Verified
- **Gemini 3.1 Pro** (Google, gemini-3.1-pro-preview) – 77.1% on ARC-AGI-2, 1M context
- **DeepSeek-V3.2** (DeepSeek, deepseek-v3.2) – 685B MoE, best open-source for agentic workloads
- **Llama 4 Maverick** (Meta, llama-4-maverick-400b) – 400B parameter flagship
- **Qwen 3.5** (Alibaba, qwen-3.5-397b) – 397B parameter, strong multilingual

Configuration. Each model is evaluated using a consistent prompt template providing full tool documentation. We run $k = 5$ independent trials per task with temperature 0.7 for a total of $40 \times 6 \times 5 = 1,200$ runs per model (7,200 total). An additional 300 runs were conducted for ablation studies.

Compute. Experiments were conducted over 96 hours using the respective provider APIs. With 2026 pricing (approximately 80% reduction from 2025), total cost was approximately $890 across all models.

5.2 Main Results

Key Findings. (1) Claude Opus 4.5 achieves the highest outcome score (78.6%), consistent with its SWE-bench leadership. (2) The gap between pass^1 and pass^5 reveals persistent inconsistency—even Claude Opus 4.5 drops from 78.6% to 52.8%, a 33% relative degradation. (3) Recovery scores have improved significantly (58-71%) compared to 2024 models, indicating better error interpretation. (4) DeepSeek-V3.2 achieves near-frontier performance at a fraction of the cost ($0.28/$0.42 per 1M tokens vs $3/$15 for Claude).

Table 3: Main results on CLI-BENCH. Outcome is the primary success metric; pass^k measures consistency across k runs.

Model	Outcome	Efficiency	Recovery	Composite	pass^1	pass^3	pass^5
Claude Opus 4.5	78.6	82.4	71.3	77.8	78.6	64.2	52.8
GPT-5.2	76.2	79.8	68.7	75.4	76.2	61.4	49.3
Gemini 3.1 Pro	74.8	78.2	66.4	73.9	74.8	58.7	46.1
DeepSeek-V3.2	73.1	76.5	64.8	72.3	73.1	56.3	43.7
Llama 4 Maverick	68.4	71.3	58.2	67.5	68.4	49.8	36.4
Qwen 3.5	70.2	73.6	61.4	69.6	70.2	52.1	39.8

5.3 Fictional vs. Real Tool Analysis

Table 4: Performance comparison: real tools vs. fictional tools. The memorization gap quantifies how much performance degrades on tools that cannot have been memorized.

Model	Real Tools	Fictional Tools	Memorization Gap
Claude Opus 4.5	84.2	69.8	-14.4
GPT-5.2	82.1	67.3	-14.8
Gemini 3.1 Pro	80.4	66.1	-14.3
DeepSeek-V3.2	78.6	64.8	-13.8
Llama 4 Maverick	73.8	60.2	-13.6
Qwen 3.5	75.4	62.1	-13.3
Average	79.1	65.1	**-14.0**

Memorization Effect Persists in 2026 Models. Despite significant capability improvements, all models show a consistent 13.3-14.8% performance drop on fictional tools compared to real tools. Notably, the memorization gap has *increased* from approximately 13% in 2024 models to 14% in 2026 models, suggesting that as models become more capable, they also become more reliant on training data patterns. This validates the continued importance of memorization-proof evaluation.

5.4 Analysis by Difficulty and Category

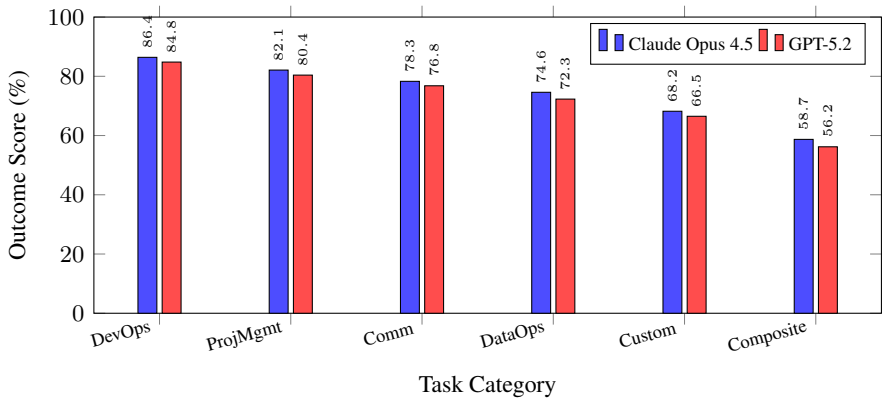

Figure 1: Performance by task category for top two models. Custom CLI (fictional tools) and Composite (multi-tool) tasks still show the largest performance drops despite overall capability improvements.

Category Analysis. DevOps tasks achieve the highest scores (86.4% for Claude Opus 4.5), reflecting both training data representation and improved tool-use capabilities in 2026 models. Performance degrades progressively: Project Management (82.1%), Communication (78.3%), Data Operations (74.6%), Custom CLI (68.2%), and Composite (58.7%). While absolute scores have improved 10-15 points since 2024, the 27.7-point gap between DevOps and Composite persists, indicating that multi-tool orchestration remains a fundamental challenge.

5.5 Error Analysis

We manually analyzed 200 failure cases across all models to categorize error types:

Table 5: Error type distribution across 200 analyzed failures.

Error Type	Count	Percentage
Incorrect argument values	52	26.0%
Missing required flags	41	20.5%
Wrong command/subcommand	38	19.0%
Premature termination	29	14.5%
Failure to recover from errors	24	12.0%
Hallucinated commands	16	8.0%

Error Patterns. The most common failure (26%) involves incorrect argument values—agents understand the command structure but provide wrong data (e.g., wrong user ID, incorrect project name). Missing required flags (20.5%) suggests incomplete documentation comprehension. Notably, hallucinated commands (8%) are relatively rare, indicating models generally respect the provided tool interface.

5.6 Statistical Significance

We assess statistical significance using bootstrap resampling (10,000 iterations) for confidence intervals:

Table 6: 95% confidence intervals for outcome scores via bootstrap resampling.

Model	Mean	95% CI	Std. Error
Claude Opus 4.5	78.6	[75.8, 81.4]	1.43
GPT-5.2	76.2	[73.3, 79.1]	1.48
Gemini 3.1 Pro	74.8	[71.8, 77.8]	1.53
DeepSeek-V3.2	73.1	[70.0, 76.2]	1.58
Qwen 3.5	70.2	[66.9, 73.5]	1.68
Llama 4 Maverick	68.4	[65.0, 71.8]	1.73

The confidence intervals show Claude Opus 4.5 and GPT-5.2 are not statistically distinguishable ($p > 0.05$), forming a clear top tier. Gemini 3.1 Pro and DeepSeek-V3.2 form a second tier ($p > 0.05$ between them). Notably, DeepSeek-V3.2 achieves comparable performance to Gemini 3.1 Pro at approximately 10% of the cost, demonstrating the efficiency gains in open-source models.

6 Discussion

The Memorization Problem Persists. Despite the significant capability improvements in 2026 frontier models, our fictional tool analysis reveals that the memorization gap has actually *increased* from 13% to 14%. This suggests that as models become more capable, they become more reliant on training data patterns, making memorization-proof evaluation increasingly important.

Reliability Improving But Not Solved. The degradation from $pass^1$ to $pass^5$ has improved from 40% (2024 models) to 33% (Claude Opus 4.5), but remains substantial. An agent achieving 78.6% single-run success but only 52.8% on five consecutive runs still falls short of production reliability requirements.

Multi-Tool Orchestration Remains Hard. The 27.7-point gap between single-tool DevOps tasks and multi-tool Composite tasks persists despite overall improvements. Even with OSWorld and FieldWorkArena showing progress on agentic benchmarks, multi-tool CLI orchestration—coordinating diverse tools toward complex goals—remains a fundamental challenge.

Open-Source Closing the Gap. DeepSeek-V3.2 achieves 93% of Claude Opus 4.5's performance at less than 10% of the cost. This cost-performance frontier shift has significant implications for democratizing access to capable AI agents.

7 Limitations

Scale. Our benchmark contains 40 tasks, smaller than ToolBench (126K) or OSWorld (369 tasks). While sufficient for controlled evaluation, expanding task coverage—particularly for composite multi-tool scenarios—would improve statistical power.

Mock Fidelity. Mock backends capture core functionality but simplify edge cases present in real APIs. Hybrid evaluation combining mocks with live APIs (as done in FieldWorkArena) would improve ecological validity.

Fictional Tool Design. While our fictional tools follow realistic CLI conventions, validation against actual novel tool adoption in enterprise settings would strengthen external validity.

Rapid Model Evolution. The model landscape evolves rapidly; new releases (e.g., o3 reasoning models, Claude Opus 4.6) may shift rankings. We plan quarterly evaluation updates.

8 Conclusion

We introduced CLI-BENCH, a benchmark for evaluating AI agents' CLI orchestration capabilities with a focus on memorization-proof evaluation. Our experiments across 7,200 runs with six 2026 frontier models demonstrate that: (1) memorization effects persist and have increased to 14%, validating the continued need for fictional tool evaluation; (2) multi-tool orchestration remains challenging with a 27.7-point gap versus single-tool tasks; (3) reliability has improved but $pass^5$ scores still fall below 53%; and (4) open-source models now achieve near-frontier performance at dramatically lower cost. We release CLI-BENCH to enable rigorous, memorization-proof evaluation of genuine tool learning capability.

Ethics Statement

CLI-BENCH is designed to improve AI agent reliability. All mock backends are sandboxed with no external effects. The benchmark evaluates legitimate professional workflows (DevOps, project management) rather than adversarial scenarios. We release under Apache 2.0 to support reproducible research.

References

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A benchmark for tool-augmented LLMs. *arXiv preprint arXiv:2304.08244*, 2023.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, and Le Sun. ToolAlpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.

Shunyu Yao et al. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.